

gemini-conversation

Exported on: 5/12/2026, 12:14:03 AM

User

Chapter No. 14 - Vector Space Classification (Till article 14.3)
now start for chapter 14 and go topic by topic as earleir

Gemini

Welcome to class, Rayyan! We are moving from the probabilistic world of Language Models into the geometric world of **Vector Space Classification**.

In Chapter 12, we looked at how likely a document was to *generate* a query. Now, we are going to look at where documents "live" in a high-dimensional space and use their positions to decide which category they belong to.



14.1 Document Representations and Measures of Relatedness

The fundamental shift here is that we treat classification as a **geometric problem**. Instead of sequences of words, we see documents as points (or vectors) in a space where each dimension corresponds to a term in our vocabulary.



1. The Document Vector

Just like we did for the Vector Space Model (VSM) in Chapter 6, we represent each document d as a vector $v(d)$.

- **Weights:** We typically use **length-normalized tf-idf weights**.



- **Why normalized?** Because we want to compare documents based on their content, not their length. If we don't normalize, a very long document would be "far" from a short document even if they discuss the exact same topic.



- **Dimensions:** If our vocabulary has M terms, our space has M dimensions.

2. The Contiguity Hypothesis (Important for Exams)

This is the "Golden Rule" of vector space classification.

Intuition: If you have a bunch of documents about "Sports" and another bunch about "Politics," the sports documents should "cluster" together in one area of the space, and the politics documents should cluster in another.



Formal Definition: Documents in the same class form a **contiguous region**, and regions of different classes do not overlap.



Exam Tip: You might be asked: "What happens if the Contiguity Hypothesis fails?"

- **Answer:** If documents from the same class are scattered all over the space (perhaps because we chose a poor weighting scheme or didn't remove stop words), then vector space classifiers like Rocchio or kNN will fail to work accurately.



3. Measuring "Relatedness"

How do we know if a new, unknown document is "related" to a class? We look at its position relative to the known training documents.

- **Similarity:** We can use **Cosine Similarity** (the angle between vectors).



- **Distance:** We can use **Euclidean Distance** (the straight-line distance between points).



For length-normalized vectors, these two are essentially equivalent for ranking purposes: as cosine similarity increases, Euclidean distance decreases.



Comparison with Chapter 13 (Naive Bayes)

- **Naive Bayes:** Uses a "bag of words" and counts (probabilistic).



- **Vector Space Classification:** Uses real-valued weights and positions (geometric).



Professor's Note: Remember, the choice of document representation is *everything*. If you use a stop list that removes the word "I," you might lose the ability to distinguish between "Personal Blogs" and "News Reports".



Does the idea of documents as points in a "neighborhood" make sense? When you're ready to learn our first classification algorithm—the **Rocchio Classifier**—just say "**next**".

User

next when explaining any topic, give its number too from book

Gemini

Great! Let's move forward to the first actual classification algorithm in the geometric world. This topic is central to understanding how IR systems categorize documents.

14.2 Rocchio Classification

Important for Exams: Rocchio is often tested because it is the simplest vector space classifier. It is a **centroid-based** method.

**The Core Intuition: The "Average" Document**

Imagine you have 10 documents about "Football" and 10 about "Tennis." Instead of looking at all 20 documents every time you want to classify a new one, wouldn't it be easier to just create one "representative" document for Football and one for Tennis?

This representative vector is called a **Centroid**. In geometry, the centroid is the "center of mass" of a set of points.

**1. Training (The Formula)**

During training, for each class c , we compute its centroid $\mu(c)$.

Formula:

$$\mu(c) = \frac{1}{|D_c|} \sum_{d \in D_c} v(d)$$



What the variables mean:

- $\mu(c)$: The centroid vector for class c .



- D_c : The set of all training documents that belong to class c .



- $|D_c|$: The total number of documents in that class.



- $v(d)$: The vector representation of a single document d (usually using tf-idf weights).



Intuition: You are literally adding up all the document vectors in a category and dividing by the number of documents to find their average position in the space.

**2. Classification (The Rule)**

When a new test document d arrives, we simply calculate its distance to each class centroid. We assign it to the class whose centroid is **closest**.



The Rule:

$$\gamma(d) = \arg \min_c |\mu(c) - v(d)|$$



- The $|x - y|$ notation represents **Euclidean Distance**.
- Alternatively, you can use **Cosine Similarity** and pick the centroid with the *maximum* similarity.



14.2.1 Decision Boundaries (Voronoi Tessellation)

What does the "border" between two classes look like in Rocchio?

Since we are picking the closest centroid, the boundary between any two classes is a **hyperplane** (a straight line in 2D, a flat plane in 3D). This line is exactly halfway between the two centroids and perpendicular to the line connecting them.



The Voronoi Tessellation: When you have multiple classes, the centroids partition the entire space into regions. Each region contains all the points closer to one specific centroid than any other. This geometric pattern is called a **Voronoi tessellation**.



14.2.2 Properties and Limitations

Strengths:

- **Speed:** It is extremely fast at test time. You only need to compare the new document to J centroids (where J is the number of classes), not to thousands of individual documents.



Weaknesses (Very Important for Exams):

1. **Spherical Assumption:** Rocchio assumes that classes are shaped like **spheres** with similar radii.
2. **Non-Contiguous Classes:** If a class is "split" into two different regions of the space (e.g., "China" might cover both ancient history and modern economy, which use very different words), the centroid will end up in the middle—possibly in a region that belongs to another class!
3. **No Training Update:** It doesn't "learn" from its mistakes; it just calculates an average.



Comparison Checklist

- **Rocchio vs. VSM:** VSM uses vectors for ranking; Rocchio uses them for **classification** (grouping).
- **Rocchio vs. Naive Bayes:** Both are **linear classifiers**. Naive Bayes is probabilistic (counting words), while Rocchio is geometric (measuring distances).

[🔗 +1](#)

- **Rocchio vs. kNN:** Rocchio is a "global" method (uses one average per class), while kNN is a "local" method (looks at individual neighbors).

[🔗](#)

Common Student Mistake: Forgetting that centroids are **not** length-normalized. While the original document vectors might be unit vectors (length 1), the average of those vectors (the centroid) will usually have a length less than 1.

[🔗](#)

Ready for our next classification algorithm, the **k-Nearest Neighbor (kNN)**? It fixes many of Rocchio's weaknesses. Just say "**next**" when you are ready!

User

next

Gemini

Excellent, let's move to the next major algorithm in the vector space family. If Rocchio was about "averages," this next method is about "neighborhoods."

14.3 k Nearest Neighbor (kNN) Classification

While Rocchio reduces an entire class to a single point (the centroid), **kNN** is a "memory-based" or "lazy" learner. It doesn't summarize the data during training; it keeps all training documents as they are and only does the work when a new query comes in.

14.3.1 The Algorithm (How it works)

To classify a new test document d :

1. **Find the Neighbors:** We look at the entire training set and find the k documents that are most similar to d (using Cosine Similarity or Euclidean Distance).
2. **The Vote:** We look at the classes of those k neighbors.
3. **Assign:** We assign d to the class that has the most "votes" among those k neighbors.

The Formula (Decision Rule): We want to find the class c that maximizes the number of its members in the neighborhood:

$$\text{class}(d) = \arg \max_{c \in C} \sum_{d_i \in S_k} I(L(d_i) = c)$$

Variable Definitions:

- S_k : The set of the k nearest neighbors of d .
- $L(d_i)$: The label (class) of neighbor d_i .
- $I(\dots)$: The indicator function (it returns 1 if the condition is true, 0 otherwise).

Intuition: If you set $k = 5$, and you find that out of the 5 closest documents to your new file, 4 are "Politics" and 1 is "Sports," you label the new file as "Politics."

14.3.2 Why kNN is "Non-Linear" (Important for Exams)

This is a crucial comparison point.

- **Rocchio and Naive Bayes** are **Linear Classifiers**. They draw a single straight line (or hyperplane) to separate classes.
- **kNN** is a **Non-Linear Classifier**. Because it looks at local neighborhoods, the "border" between classes can be very wiggly, complex, and curved.

Numerical Intuition: Imagine a class shaped like a "C" wrapped around another class. A straight line (Rocchio) would fail to separate them because the centroid of the "C" would likely land inside the other class. But kNN doesn't care about the shape; it only cares about who is next door. It can follow the curve of the "C" perfectly.

14.3.3 Choosing the value of k

Choosing k is the most important part of using this algorithm.

- **If $k = 1$:** The decision boundary is very "sensitive" to noise. One misplaced document (an outlier) can create a tiny "island" of the wrong class inside a correct one. (Overfitting).
- **If k is very large:** The boundary becomes smoother and starts to look more like Rocchio. If $k = N$ (total documents), every document will just be assigned to the most common class in the whole dataset! (Underfitting).

Exam Tip: Usually, k is an odd number (3, 5, 7) to avoid "ties" in voting when you have two classes.

Numerical Step-by-Step Example

Scenario: Suppose we use **Euclidean Distance** and $k = 3$. We have a new document X and the following 4 training points:

- 1. d_1 : Distance = 1.2, Class = **Sports**
- 2. d_2 : Distance = 0.5, Class = **Politics**
- 3. d_3 : Distance = 2.0, Class = **Sports**
- 4. d_4 : Distance = 0.8, Class = **Politics**

Step 1: Sort by Distance (Closest to Furthest)

- 1. d_2 (0.5)
- 2. d_4 (0.8)
- 3. d_1 (1.2)
- 4. d_3 (2.0)

Step 2: Pick the top $k = 3$ neighbors The neighbors are d_2, d_4, d_1 .

Step 3: Count the Votes

- Politics: 2 votes (d_2, d_4)
- Sports: 1 vote (d_1)

Result: Document X is classified as **Politics**.

Comparison: Rocchio vs. kNN (Article 14.3 Comparison)

Feature	Rocchio (14.2)	kNN (14.3)
Model Type	Centroid-based (Global)	Neighborhood-based (Local)
Boundary	Linear (Straight)	Non-linear (Complex/Wiggly)
Training Speed	Slow (computing averages)	Instant (just store data)
Test Speed	Fast (compare to centroids)	Slow (compare to every doc)
Memory	Low (only store centroids)	High (store every doc)

 Export to Sheets

Common Student Mistake: Thinking that a larger k always means better accuracy. In reality, a very large k can ignore local patterns and lead to "misclassifying" small but distinct clusters of data.

Professor's Conclusion: We have now reached the end of Section 14.3 as you requested! You now know the two main ways to use vector spaces for classification: the "Average" approach (Rocchio) and the "Neighbor" approach (kNN).

Since you asked to go "Till article 14.3," this is our stopping point for this chapter. Do you have any questions on the math of kNN or the comparisons, or would you like to move to a new topic or chapter?

User

explain 14.3 in detail

Gemini

14.3 k Nearest Neighbor (kNN)

In our previous lesson, we saw how the **Rocchio Classifier** summarizes an entire class into a single "average" point (the centroid). While fast, it fails if a class isn't a perfect sphere.

🔗 +2

Now, we will learn about **k Nearest Neighbor (kNN)**, which is a "memory-based" or "lazy" learner. Instead of summarizing the data, kNN looks at the actual documents in the "neighborhood" of a new document to decide its class.

🔗 +3

14.3.1 The Intuition: "Birds of a Feather"

The rationale for kNN is the **Contiguity Hypothesis**: we expect a test document d to have the same label as the training documents located in its local region.

🔗 +1

Imagine you are in a room full of people. If the three people standing closest to you are all "Computer Science students," there is a very high probability that you are also a "Computer Science student." kNN applies this exact logic to document vectors in space.

🔗 +1

The Basic Decision Rules

1. **1NN (1-Nearest Neighbor)**: Assign the test document to the class of its single closest neighbor in the training set.

🔗 +1

2. **kNN**: Find the k closest neighbors and assign the test document to the **majority class** among them.

🔗 +1

14.3.2 Decision Boundaries: Voronoi Tessellation

Important for Exams: The decision boundaries in 1NN are segments of a **Voronoi Tessellation**.

🔗 +1

- A **Voronoi Cell** is a region where every point inside it is closer to one specific training document than to any other.

🔗 +1

- The space is partitioned into convex polygons.

🔗 +1

- **Why it matters:** Unlike the straight lines of Rocchio, kNN's boundaries can be complex, wiggly, and follow the specific shape of the data.

🔗 +1

14.3.3 The kNN Algorithm and Formulas

Probabilistic kNN: We can estimate the probability that document d belongs to class c as the fraction of the k neighbors that have that label:

🔗 +1

$$\hat{P}(c|d) = \frac{|S_k \cap c|}{k}$$

- S_k : The set of k nearest neighbors of d .

🔗

- $|S_k \cap c|$: The number of those neighbors that belong to class c .

🔗

Weighted kNN (Advanced): Simple voting can be "dumb." What if the closest neighbor is very close, but the other two are far away? We can weight the "votes" by their **Cosine Similarity**:

🔗

$$\text{score}(c, d) = \sum_{d' \in S_k} I_c(d') \cdot \cos(v(d'), v(d))$$

- $I_c(d')$: An indicator function that is 1 if d' is in class c , and 0 otherwise.

🔗

- The class with the highest total score wins.

🔗

🔪 Example 14.2: Step-by-Step Numerical

Suppose we have a test document d_5 and four training documents (d_1, d_2, d_3, d_4) .

🔗

1. **Calculate Distances:** We find the Euclidean distances between the test doc and the training docs:

- $|d_1 - d_5| \approx 1.41$
- $|d_2 - d_5| \approx 1.41$
- $|d_3 - d_5| \approx 1.41$
- $|d_4 - d_5|[\text{cite}_s \text{tart}] = 0.0$

🔗

2. **Find the Nearest Neighbor:** For 1NN, the nearest neighbor is d_4 (distance 0.0).

🔗

3. **Assign Class:** If d_4 is in class C , then 1NN assigns d_5 to class C .

🔗

Time Complexity: Training vs. Testing

This is a frequent exam comparison. Note how kNN "flips" the work compared to Rocchio.

🔗 +2

Phase	kNN Complexity	Logic
Training	$\Theta(1)$	We do almost nothing! Just store the documents. 🔗
Testing	$\Theta(D \cdot L_{ave})$	D

📄 Export to Sheets



- $|D|$: Total number of documents in the training set.
🔗
- L_{ave} : Average length of a document.
🔗 +1

Exam Tip: kNN is called a **Lazy Learner** because it postpones all the computational work until a test document needs to be classified.

🔗

Comparison Summary for Section 14.3

- **Rocchio:** Global, fast testing, assumes spherical classes.
🔗 +1
- **kNN:** Local, slow testing, can handle any complex class shape (non-spherical, disconnected).
🔗 +1
- **Naive Bayes:** Probabilistic, linear, while kNN is geometric and non-linear.
🔗

We have now finished Section 14.3. This marks the end of the topics you requested for Chapter 14. Do you have any questions on the Voronoi boundaries or the complexity math, or would you like to review anything from Chapters 12 or 14?

User

now I want you to solve me numerical one same first with rochio and then with KNN, while explaining me and using concepts discussed earlier. Use book question

Gemini

Let's put everything we've learned into practice with a concrete numerical example. We will use the data from **Example 14.1 and 14.2** in your textbook. This is a classic "China" vs. "Not-China" classification problem.

🔗 +1

The Dataset

We have four training documents (d_1 to d_4) and one test document (d_5). The documents are represented as vectors using six terms: **Chinese, Japan, Tokyo, Macao, Beijing, Shanghai**.

🔗

Training Set:

- **Class c (China):**
 - $d_1 = [0, 0, 0, 0, 1.0, 0]$
 - $d_2 = [0, 0, 0, 0, 0, 1.0]$
 - $d_3 = [0, 0, 0, 1.0, 0, 0]$
- **Class $\bar{c}$ (Not China):**
 - $d_4 = [0, 0.71, 0.71, 0, 0, 0]$

Test Document (To be classified):

- $d_5 = [0, 0.71, 0.71, 0, 0, 0]$

Part 1: Rocchio Classification

Recall the intuition: Rocchio builds a "prototype" (centroid) for each class and compares the test document to these averages.

🔗

Step 1: Calculate Class Centroids (μ)

The centroid is the average of all document vectors in that class: $\mu(c) = \frac{1}{|D_c|} \sum v(d)$.

🔗

- **Centroid for Class c (China):**

$$\mu_c = \frac{d_1 + d_2 + d_3}{3} = \frac{[0, 0, 0, 1, 1, 1]}{3} = [0, 0, 0, 0.33, 0.33, 0.33]$$

- **Centroid for Class $\bar{c}$ (Not China):**

$$\mu_{\bar{c}} = \frac{d_4}{1} = [0, 0.71, 0.71, 0, 0, 0]$$

Step 2: Calculate Euclidean Distance from d_5 to Centroids

We use the formula: $\text{Dist} = \sqrt{\sum (x_i - y_i)^2}$.



- **Distance to μ_c :**

$$|\mu_c - d_5| = \sqrt{(0 - 0)^2 + (0 - 0.71)^2 + (0 - 0.71)^2 + (0.33 - 0)^2 + (0.33 - 0)^2 + (0.33 - 0)^2}$$

$$= \sqrt{0 + 0.5 + 0.5 + 0.11 + 0.11 + 0.11} = \sqrt{1.33} \approx 1.15$$

- **Distance to $\mu_{\bar{c}}$:**

$$|\mu_{\bar{c}} - d_5| = \sqrt{(0 - 0)^2 + (0.71 - 0.71)^2 + (0.71 - 0.71)^2 + (0 - 0)^2 + (0 - 0)^2 + (0 - 0)^2}$$

$$= \sqrt{0} = 0.0$$

Decision:

Since $0.0 < 1.15$, d_5 is closer to $\mu_{\bar{c}}$. **Rocchio assigns d_5 to class $\bar{c}$ (Not China).**



Part 2: k-Nearest Neighbor (kNN) Classification

Recall the intuition: kNN doesn't average anything; it looks at the actual individuals nearby.



Step 1: Calculate Distance to All Training Points

We need the distance between d_5 and every document in the training set.



1. $|d_1 - d_5| = \sqrt{0 + (0.71)^2 + (0.71)^2 + 0 + (1)^2 + 0} = \sqrt{0.5 + 0.5 + 1} = \sqrt{2} \approx 1.41$
2. $|d_2 - d_5| = \sqrt{0 + (0.71)^2 + (0.71)^2 + 0 + 0 + (1)^2} = \sqrt{2} \approx 1.41$
3. $|d_3 - d_5| = \sqrt{0 + (0.71)^2 + (0.71)^2 + (1)^2 + 0 + 0} = \sqrt{2} \approx 1.41$
4. $|d_4 - d_5| = \sqrt{0 + 0 + 0 + 0 + 0 + 0} = 0.0$

Step 2: Apply Decision Rule for $k = 1$ (1NN)

- **Neighbors:** The single closest neighbor is d_4 .
- **Class of d_4 :** $\bar{c}$.
- **Assignment:** 1NN assigns d_5 to class $\bar{c}$.



Step 3: Apply Decision Rule for $k = 3$ (3NN)

- **Neighbors:** The 3 closest are d_4 (dist 0) and any two of $\{d_1, d_2, d_3\}$ (dist 1.41).
- **Votes:**
 - Class $\bar{c}$: 1 vote (d_4)
 - Class c : 2 votes (e.g., d_1 and d_2)

- **Assignment:** 3NN assigns d_5 to class c (China).



Analysis & Interpretation

This example beautifully illustrates the difference between these two methods:

1. **Global vs. Local:** Rocchio (c) and 1NN ($\bar{c}$) made the same decision, but 3NN (c) flipped it.



2. **Robustness:** 1NN is very sensitive to a single outlier. In this case, d_5 and d_4 are identical, so 1NN is "convinced" they are the same.



3. **Linear vs. Non-linear:** Rocchio assumes class c is a sphere around μ_c . 3NN shows that when you expand the neighborhood, the "majority" influence of the three China documents takes over, even though one Not-China document is closer.



Common Student Mistake: Forgetting that kNN distances are calculated for *every* training document individually, while Rocchio only requires distances to class *centroids*.



Would you like to move to a new topic or discuss any specific step of these calculations further?

User

and solve using this too

Gemini

This is a perfect example to work through, Rayyan! It's a classic coordinate-based problem that clearly demonstrates how **Rocchio** (the "averager") and **kNN** (the "neighbor-watcher") can sometimes end up in a tight race.

Let's break down the data from your image and solve it step-by-step.

The Problem Setup

We have two features (w_1, w_2) and seven training documents.

- **Class 1 Documents:** $d_1(1, 1), d_2(1, 2), d_3(2, 1), d_4(2, 2)$
- **Class 2 Documents:** $d_5(6, 6), d_6(6, 7), d_7(7, 6)$
- **Test Instance:** $x = (4, 4)$

Part 1: Rocchio Classification

Step 1: Calculate Centroids (μ) The centroid is the average of the coordinates for each class.

- μ_1 (Class 1):

$$w_1 = \frac{1 + 1 + 2 + 2}{4} = 1.5, \quad w_2 = \frac{1 + 2 + 1 + 2}{4} = 1.5$$

$$\mu_1 = (1.5, 1.5)$$

- μ_2 (Class 2):

$$w_1 = \frac{6 + 6 + 7}{3} \approx 6.33, \quad w_2 = \frac{6 + 7 + 6}{3} \approx 6.33$$

$$\mu_2 = (6.33, 6.33)$$

Step 2: Calculate Euclidean Distance from $x(4, 4)$ to Centroids Using the formula: $d(x, y) = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2}$

- **Distance to μ_1 :**

$$d(x, \mu_1) = \sqrt{(4 - 1.5)^2 + (4 - 1.5)^2} = \sqrt{2.5^2 + 2.5^2} = \sqrt{6.25 + 6.25} = \sqrt{12.5} \approx 3.54$$

- **Distance to μ_2 :**

$$d(x, \mu_2) = \sqrt{(4 - 6.33)^2 + (4 - 6.33)^2} = \sqrt{(-2.33)^2 + (-2.33)^2} \approx \sqrt{5.43 + 5.43} \approx \sqrt{10.86} \approx 3.30$$

Decision: Since $3.30 < 3.54$, the test instance x is closer to the Class 2 centroid. **Rocchio Result: Class 2**

Part 2: k-Nearest Neighbor (kNN) Classification

Step 1: Calculate Distances from $x(4, 4)$ to all points

$$1. d_1(1, 1) : \sqrt{3^2 + 3^2} = \sqrt{18} \approx 4.24$$

$$2. d_2(1, 2) : \sqrt{3^2 + 2^2} = \sqrt{13} \approx 3.61$$

$$3. d_3(2, 1) : \sqrt{2^2 + 3^2} = \sqrt{13} \approx 3.61$$

$$4. d_4(2, 2) : \sqrt{2^2 + 2^2} = \sqrt{8} \approx \mathbf{2.83 \text{ (Class 1)}}$$

$$5. d_5(6, 6) : \sqrt{2^2 + 2^2} = \sqrt{8} \approx \mathbf{2.83 \text{ (Class 2)}}$$

$$6. d_6(6, 7) : \sqrt{2^2 + 3^2} = \sqrt{13} \approx 3.61$$

$$7. d_7(7, 6) : \sqrt{3^2 + 2^2} = \sqrt{13} \approx 3.61$$

Step 2: Sorting and Voting The distances reveal a fascinating "tie" for the closest position!

- **If $k = 1$:** There is a tie between d_4 (Class 1) and d_5 (Class 2). In an exam, if this happens, you should mention the tie and typically use a tie-breaker (like picking the class with the higher overall frequency or a random choice).
- **If $k = 3$:** Our neighbors are d_4, d_5 , and then we have a four-way tie for the 3rd neighbor (d_2, d_3, d_6, d_7).
 - If we pick d_2 or $d_3 \rightarrow$ **Class 1 Wins** (2 vs 1).
 - If we pick d_6 or $d_7 \rightarrow$ **Class 2 Wins** (2 vs 1).

Professor's Interpretation & Exam Tips

1. **The "Borderline" Case:** Notice that $x(4, 4)$ is almost exactly in the middle of the two groups. Rocchio leans toward Class 2 because Class 2 is slightly more "compact" and its centroid is pulled closer to the center than the 4-point Class 1.
2. **Tie-Breaking:** If you see a tie like this in an exam, **explain it**. Write: "*Since d_4 and d_5 are equidistant, the classification is ambiguous for $k = 1$.*"
3. **Visualization:** If you plotted these, you'd see two squares of points. $x(4, 4)$ is the "bridge" between them.
4. **Common Mistake:** Don't round too early! If I had rounded 6.33 to just 6, the Rocchio distance would have looked much smaller than it actually is. Always keep at least two decimal places during your work.

Does this numerical walkthrough for your specific table make sense? We've reached the end of the sections you asked for—is there anything else from these chapters you'd like to dive into, or are you feeling ready for the exam?

User

who will calculate I_c ? solve the whole question man

Gemini

I hear you—let’s stop skipping steps and solve this exactly as you would on a final exam. I will act as the "calculator" for I_c and show you how it functions as a logic gate.

To solve the whole question using the formula $\text{score}(c, d) = \sum_{d' \in S_k} I_c(d') \cos(v(d'), v(d))$, we must follow a rigid 4-step process.

Step 1: Pre-calculate the Test Vector Magnitude

Before we can find the similarity for any neighbor, we need the length (magnitude) of our test document $x(4, 4)$.

$$\|x\| = \sqrt{4^2 + 4^2} = \sqrt{16 + 16} = \sqrt{32} \approx 5.657$$

Step 2: Calculate Similarity for ALL Training Points

To find the top $k = 3$ neighbors, we must calculate the cosine similarity for every point in the table. *Note:* $\cos = \frac{x_1y_1 + x_2y_2}{\|x\| \cdot \|d_i\|}$

Document	Coordinates	Magnitude ($\ d_i\ $)	Dot Product ($x \cdot d_i$)
d_1	(1, 1)	$\sqrt{1^2 + 1^2} = 1.414$	$(4 \cdot 1) + (4 \cdot 1) = 8$
d_2	(1, 2)	$\sqrt{1^2 + 2^2} = 2.236$	$(4 \cdot 1) + (4 \cdot 2) = 12$
d_3	(2, 1)	$\sqrt{2^2 + 1^2} = 2.236$	$(4 \cdot 2) + (4 \cdot 1) = 12$
d_4	(2, 2)	$\sqrt{2^2 + 2^2} = 2.828$	$(4 \cdot 2) + (4 \cdot 2) = 16$
d_5	(6, 6)	$\sqrt{6^2 + 6^2} = 8.485$	$(4 \cdot 6) + (4 \cdot 6) = 48$
d_6	(6, 7)	$\sqrt{6^2 + 7^2} = 9.220$	$(4 \cdot 6) + (4 \cdot 7) = 52$

Document	Coordinates	Magnitude ($\ d_i\ $)	Dot Product ($x \cdot d_i$)
d_7	(7, 6)	$\sqrt{7^2 + 6^2} = 9.220$	$(4 \cdot 7) + (4 \cdot 6) = 52$



Export to Sheets



Step 3: Identify the Top $k = 3$ Neighbors (S_3)

We look at the "Cosine Similarity" column and pick the three highest values:

- 1. d_1 : Similarity = 1.0 (Class 1)
- 2. d_4 : Similarity = 1.0 (Class 1)
- 3. d_5 : Similarity = 1.0 (Class 2)

So, our set $S_3 = \{d_1, d_4, d_5\}$.

Step 4: Final Score Calculation (The I_c Stage)

This is where we solve the formula for each class. $I_c(d')$ is the "Indicator Function"—it is **1** if the neighbor belongs to the class we are calculating, and **0** if it doesn't.

Score for Class 1:

$$\text{Score}(1, x) = [I_1(d_1) \cdot \cos(d_1, x)] + [I_1(d_4) \cdot \cos(d_4, x)] + [I_1(d_5) \cdot \cos(d_5, x)]$$

- d_1 is Class 1 $\rightarrow I_1 = 1$
- d_4 is Class 1 $\rightarrow I_1 = 1$
- d_5 is Class 2 $\rightarrow I_1 = 0$

$$\text{Score}(1, x) = (1 \cdot 1.0) + (1 \cdot 1.0) + (0 \cdot 1.0) = 1.0 + 1.0 + 0 = \mathbf{2.0}$$

Score for Class 2:

$$\text{Score}(2, x) = [I_2(d_1) \cdot \cos(d_1, x)] + [I_2(d_4) \cdot \cos(d_4, x)] + [I_2(d_5) \cdot \cos(d_5, x)]$$

- d_1 is Class 1 $\rightarrow I_2 = 0$
- d_4 is Class 1 $\rightarrow I_2 = 0$
- d_5 is Class 2 $\rightarrow I_2 = 1$

$$\text{Score}(2, x) = (0 \cdot 1.0) + (0 \cdot 1.0) + (1 \cdot 1.0) = 0 + 0 + 1.0 = \mathbf{1.0}$$

Final Interpretation & Exam Tips

Result: Class 1 Wins with a score of 2.0 vs 1.0.

Who calculates I_c ? In an exam, **you** do! You simply look at the training table. If you are calculating the score for Class 1, you "turn off" (multiply by 0) any neighbor that isn't Class 1.

Important Exam Difference:

- **Rocchio** (which we did earlier) chose **Class 2** because $x(4, 4)$ was physically closer to the Class 2 cluster in terms of distance.
- **Weighted kNN** (using Cosine) chose **Class 1** because even though the points are further away, there are *more* points in Class 1 that point in the exact same direction as our test document.

Common Mistake: Forgetting that k limits your view. Students often try to calculate I_c for all 7 documents. You **must** only use the k neighbors you identified in Step 3.

Does this full breakdown clear up the confusion around I_c and the final scoring? Ready for the next topic, or should we try another one of these?

User

so we first filter out on the k highest cosine values?

Gemini

Exactly right, Rayyan! You've got the sequence perfectly. In the world of kNN, your first job is to be a "bouncer" at a club: you only let in the k most similar individuals.

Here is the precise "Exam-Style" logical flow for that filtering process.

The "Filter First" Logic

Before you even look at the I_c formula, you must isolate your neighborhood. This is a 2-step filtering process:

1. **The Calculation:** You compute the similarity for **every single document** in your training set against the test document. (If there are 1,000 documents, you do 1,000 calculations).
2. **The Ranking (Filtering):** You sort those results from highest similarity to lowest similarity and "cut the line" at exactly k .

Important for Exams: Only the documents that survive this cut are allowed to "vote" in the I_c formula. Even if a document is 99% similar, if it is the $(k + 1)$ -th document, its I_c value is never even checked—it is ignored.

Why we filter first (Intuition)

If we didn't filter and instead applied the formula to the whole collection, the "majority" would always win simply by having more documents, regardless of how irrelevant they are. By filtering for the k highest values, we ensure we are only listening to the **local expertise** of the documents most like our test case.

Numerical Walkthrough of the "Filter"

Using the table from your image again, let's look at why the filter is so critical.

Query Document: $x(4, 4)$ **Goal:** Find $k = 3$ neighbors.

1. **We calculate all similarities:**

- d_1 : 1.0
- d_4 : 1.0
- d_5 : 1.0
- d_6 : 0.997
- d_7 : 0.997
- d_2 : 0.948
- d_3 : 0.948

2. **The Filter:** We pick the top 3.

- Since d_1, d_4 , and d_5 are all **1.0**, they are our $k = 3$ neighbors.
- Documents d_6, d_7, d_2 , and d_3 are **discarded**.

3. **The Formula:** We only plug d_1, d_4 , and d_5 into the $\sum I_c \cdot \cos$ equation.

Exam Tip: The "Tie" Trap

In your specific table, notice that d_6 and d_7 have a similarity of **0.997**. They are extremely close! If your k was 5 instead of 3, they would have made the cut and completely changed the score for Class 2.

Common Student Mistake: Students sometimes see a high similarity (like 0.997) and instinctively include it in the calculation because "it seems relevant." **Don't do it!** If the question says $k = 3$, you must strictly stop after the 3rd highest value.

Summary of the Process:

1. **Calculate** all similarities.
2. **Sort** them.
3. **Filter** for the top k .
4. **Score** using the I_c formula for each class.

Does that clarify the "Filter" step? Are you ready to move on to the next topic, or should we look at how this changes if we use **Distance** instead of **Cosine**?